

Salesforce Developer Bridge Program – Day 2 Documentation

Name: G. Bhanu Durga Ram Prasad

Technology: Salesforce Development

Platform: Salesforce Trailhead Playground

Objective

The objective of Day 2 training was to strengthen Salesforce development skills by learning Apex Collections, Governor Limits and Bulkification, and Asynchronous Apex. These topics are essential for writing efficient, scalable, and production-ready Apex code.

Task 1 – Apex Collections (List, Set, Map)

Objective

The objective of this task was to understand the three primary Apex collection types—**List**, **Set**, and **Map**—and learn how they are used to efficiently store, retrieve, and process Salesforce data. The training required creating a List of city names, a Set of record IDs, and a Map of records retrieved through SOQL.

Activities Performed

1. List Collection

- Created a `List<String>` containing five city names.
- Used a **for loop** to iterate through the list.
- Converted each city name to uppercase using the `toUpperCase()` method.
- Displayed the output in the Debug Log.

2. Set Collection

- Queried records from the custom object.
- Stored the record IDs in a `Set<Id>`.

- Printed the size of the set to verify that duplicate values were automatically removed.

3. Map Collection

- Queried records from the custom object.
- Stored them in a `Map<Id, Patient__c>`.
- Retrieved a record using its Id as the key.
- Displayed one of the record fields using the retrieved object.

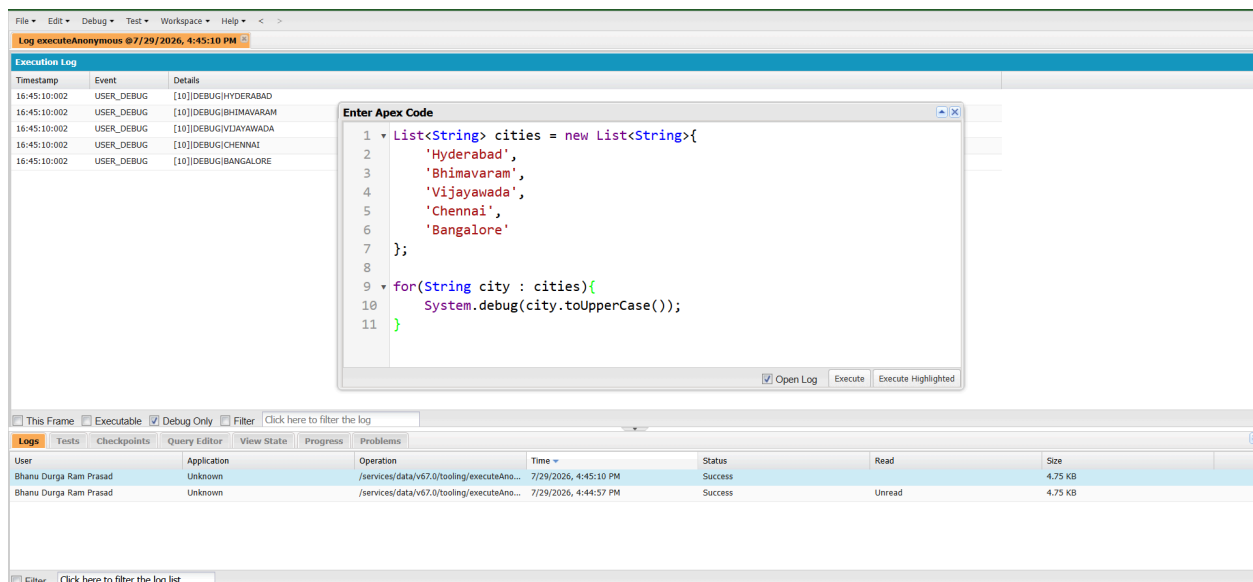
Learning Outcome

After completing this task, I learned:

- How to create and use **List**, **Set**, and **Map** collections.
- The difference between ordered and unordered collections.
- How Sets automatically eliminate duplicate values.
- How Maps provide fast access to records using unique keys.
- How collections improve Apex performance.

Result

All collection programs executed successfully in the Execute Anonymous window, and the Debug Log displayed the expected output without errors.



The screenshot displays the Salesforce IDE interface. At the top, a menu bar includes File, Edit, Debug, Test, Workspace, and Help. Below the menu, a toolbar shows 'Log executeAnonymous @7/29/2026, 4:45:10 PM'. The main workspace is divided into two panes. The left pane, titled 'Execution Log', contains a table with columns for Timestamp, Event, and Details. The right pane, titled 'Enter Apex Code', contains the following Apex code:

```
1 List<String> cities = new List<String>{
2     'Hyderabad',
3     'Bhimavaram',
4     'Vijayawada',
5     'Chennai',
6     'Bangalore'
7 };
8
9 for(String city : cities){
10     System.debug(city.toUpperCase());
11 }
```

Below the code editor, there is a filter bar with checkboxes for 'This Frame', 'Executable', 'Debug Only', and 'Filter'. The 'Debug Only' checkbox is checked. Below the filter bar, there is a table with columns for User, Application, Operation, Time, Status, Read, and Size. The table contains two rows of data:

User	Application	Operation	Time	Status	Read	Size
Bhanu Durga Ram Prasad	Unknown	/services/data/v67.0/tooling/executeAno...	7/29/2026, 4:45:10 PM	Success		4.75 KB
Bhanu Durga Ram Prasad	Unknown	/services/data/v67.0/tooling/executeAno...	7/29/2026, 4:44:57 PM	Success	Unread	4.75 KB

At the bottom of the interface, there is a filter bar with a 'Filter' checkbox and a 'Click here to filter the log list' link.

Execution Log

Timestamp	Event	Details
16:59:11:007	USER_DEBUG	[12]DEBUG Number of unique patients: 3

Enter Apex Code

```

1 List<Patient__c> patients = [
2     SELECT Id
3     FROM Patient__c
4 ];
5
6 Set<Id> patientIds = new Set<Id>();
7
8 for(Patient__c p : patients){
9     patientIds.add(p.Id);
10 }
11
12 System.debug('Number of unique patients: ' + patientIds.size());

```

☒ Open Log

☐ This Frame ☐ Executable ☒ Debug Only ☐ Filter

User	Application	Operation	Time	Status	Read	Size
Bhanu Durga Ram Prasad	Unknown	/services/data/v67.0/tooling/executeAno...	7/29/2026, 4:59:11 PM	Success		4.47 KB
Bhanu Durga Ram Prasad	Unknown	/services/data/v67.0/tooling/executeAno...	7/29/2026, 4:55:34 PM	Success		3.53 KB
Bhanu Durga Ram Prasad	Unknown	/services/data/v67.0/tooling/executeAno...	7/29/2026, 4:52:49 PM	Success		4.51 KB
Bhanu Durga Ram Prasad	Unknown	/services/data/v67.0/tooling/executeAno...	7/29/2026, 4:45:10 PM	Success		4.75 KB
Bhanu Durga Ram Prasad	Unknown	/services/data/v67.0/tooling/executeAno...	7/29/2026, 4:44:57 PM	Success	Unread	4.75 KB

Execution Log

Timestamp	Event	Details
16:59:48:016	USER_DEBUG	[8]DEBUG Girish

Enter Apex Code

```

1 Map<Id, Patient__c> patientMap =
2     new Map<Id, Patient__c>{
3         [SELECT Id, Name FROM Patient__c]
4     };
5
6 Id patientId = patientMap.keySet().iterator().next();
7
8 System.debug(
9     patientMap.get(patientId).Name
10 );

```

☒ Open Log

☐ This Frame ☐ Executable ☒ Debug Only ☐ Filter

User	Application	Operation	Time	Status	Read	Size
Bhanu Durga Ram Prasad	Unknown	/services/data/v67.0/tooling/executeAno...	7/29/2026, 4:59:48 PM	Success		3.53 KB
Bhanu Durga Ram Prasad	Unknown	/services/data/v67.0/tooling/executeAno...	7/29/2026, 4:59:11 PM	Success		4.47 KB
Bhanu Durga Ram Prasad	Unknown	/services/data/v67.0/tooling/executeAno...	7/29/2026, 4:55:34 PM	Success		3.53 KB
Bhanu Durga Ram Prasad	Unknown	/services/data/v67.0/tooling/executeAno...	7/29/2026, 4:52:49 PM	Success		4.51 KB
Bhanu Durga Ram Prasad	Unknown	/services/data/v67.0/tooling/executeAno...	7/29/2026, 4:45:10 PM	Success		4.75 KB
Bhanu Durga Ram Prasad	Unknown	/services/data/v67.0/tooling/executeAno...	7/29/2026, 4:44:57 PM	Success	Unread	4.75 KB

Task 2 – Governor Limits and Bulkification

Objective

The objective of this task was to understand Salesforce Governor Limits and learn how to optimize Apex code using Bulkification techniques. The activity involved intentionally writing an inefficient trigger to produce a Governor Limit error and then refactoring the code by moving SOQL and DML operations outside the loop.

Activities Performed

Step 1 – Created a Bad Trigger

- Wrote a trigger with a SOQL query inside a **for** loop.
- Executed the trigger by inserting multiple records.

Step 2 – Triggered Governor Limit

- Inserted more than 200 records.
- Salesforce exceeded the maximum SOQL query limit.
- The system generated the error:

"Too many SOQL queries: 101"

Step 3 – Bulkified the Trigger

To optimize the trigger:

- Moved the SOQL query outside the loop.
- Used a **List** and **Map** to process records efficiently.
- Executed the trigger again with 200+ records.

Result

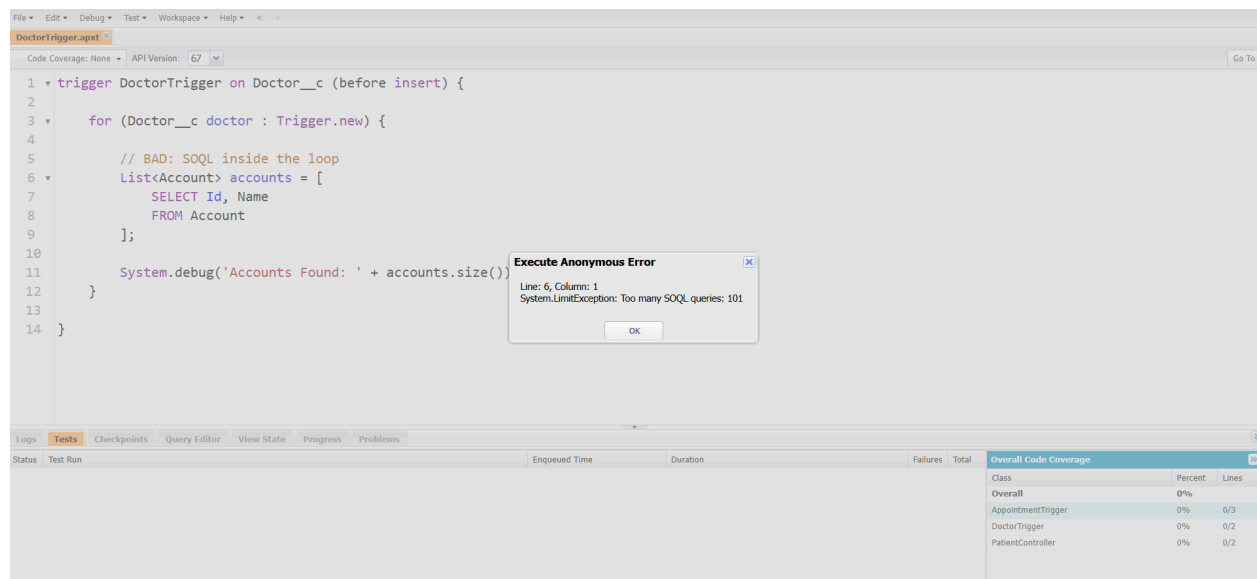
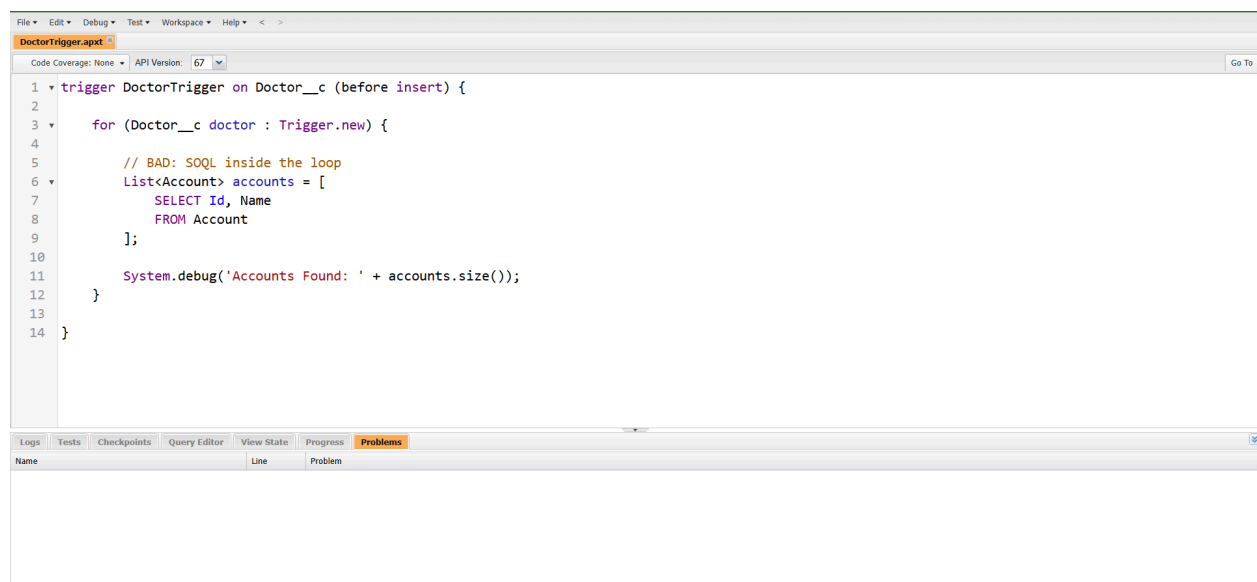
After bulkification:

- All records were processed successfully.
- No Governor Limit errors occurred.
- The trigger became scalable and suitable for production environments.

Learning Outcome

This activity helped me understand:

- Salesforce Governor Limits.
- Why SOQL should never be placed inside loops.
- How Bulkification improves Apex performance.
- How Lists and Maps reduce unnecessary database operations.
- Best practices for writing production-ready Apex triggers.





Task 3 – Asynchronous Apex (@future Method)

Objective

The objective of this task was to understand Asynchronous Apex by creating a simple **@future** method. The method receives a record Id, queries the corresponding record, updates one of its fields, and executes asynchronously. The completed job can then be verified in the **Apex Jobs** page.

Activities Performed

- Created an Apex class containing a **@future** method.
- Passed a record Id as a parameter.
- Queried the record inside the future method.
- Updated one field on the record.
- Called the future method from the Execute Anonymous window.
- Verified successful execution through **Setup** → **Apex Jobs**.

Working Process

1. A record Id is passed to the **@future** method.
2. Salesforce schedules the method for background execution.
3. The method queries the specified record.
4. The required field is updated.
5. The transaction completes asynchronously.
6. The completed job is visible in the Apex Jobs page.

Future Method vs Queueable Apex vs Batch Apex

Feature	Future Method	Queueable Apex	Batch Apex
Execution	Asynchronous	Asynchronous	Asynchronous
Parameters	Primitive data types only	Supports complex objects	Processes large datasets
Monitoring	Apex Jobs	Apex Jobs	Apex Jobs
Chaining	Not Supported	Supported	Supported
Best Use	Simple background tasks	Complex background processing	Large-volume data processing

Learning Outcome

After completing this task, I learned:

- How asynchronous processing works in Salesforce.
- How to create and execute a **@future** method.
- The differences between Future, Queueable, and Batch Apex.
- How to monitor asynchronous jobs using Apex Jobs.

Result

The future method executed successfully, updated the required record, and appeared in the **Apex Jobs** page, confirming asynchronous execution.

File Edit Debug Test Workspace Help < >

DoctorTrigger.apex Log executeAnonymous @7/29/2026, 9:08:13 PM FutureDoctorClass.apex

Code Coverage: None API Version: 67 Go To

```

1 public class FutureDoctorClass {
2
3     @future
4     public static void updateDoctor(Id doctorId) {
5
6         Doctor__c doctor = [
7             SELECT Id, Name
8             FROM Doctor__c
9             WHERE Id = :doctorId
10            LIMIT 1
11        ];
12
13        doctor.Name = doctor.Name + ' Updated';
14
15        update doctor;
16    }
17 }

```

Logs Tests Checkpoints Query Editor View State Progress Problems

Status	Test Run	Enqueued Time	Duration	Failures	Total

Overall Code Coverage		
Class	Percent	Lines
Overall	0%	
AppointmentTrigger	0%	0/3
DoctorTrigger	0%	0/2
FutureDoctorClass	0%	0/5
PatientController	0%	0/2

SETUP

Apex Jobs

[Click here to go to the new batch jobs page](#)

Apex Jobs

[Help for this Page](#)

Monitor the status of all Apex jobs, and optionally, abort jobs that are in progress.

Percent of Asynchronous Apex Used: 0%
You have currently used 1 asynchronous Apex operations out of an allowed 24-hour organization limit of 250,000. To learn about how this limit is calculated and what contributes to it, see the [Lightning Platform Apex Limits](#) topic.

View: All Create New View

Action	Submitted Date +	Job Type	Status	Status Detail	Total Batches	Batches Processed	Failures	Submitted By	Completion Date	Apex Class	Apex Method	Apex Job ID
	29/07/2026, 9:12 pm	Future	Completed		0	0	0	Ram Prasad, Bhanu Durga	29/07/2026, 9:12 pm	FutureDoctorClass	updateDoctor	707dm00001Z171z

Task 4 – Lightning Web Components (LWC) – Parent-to-Child and Child-to-Parent Communication

Objective

The objective of this task was to understand communication between Lightning Web Components (LWCs). Two components were created to demonstrate **Parent-to-Child** communication using the `@api` decorator and **Child-to-Parent** communication using custom events. This activity helped in learning how components exchange data and interact with each other in Salesforce applications.

Activities Performed

Step 1 – Created Parent Component

- Created a Parent Lightning Web Component.
 - Displayed a selected Patient record.
 - Passed the Patient Record Id to the Child component using a public `@api` property.
-

Step 2 – Created Child Component

- Created a Child Lightning Web Component.
 - Received the Record Id from the Parent component.
 - Displayed the Patient details using the received Record Id.
-

Step 3 – Child-to-Parent Communication

- Added a button in the Child component.
 - Fired a custom event when the button was clicked.
 - The Parent component listened for the event.
 - Updated the displayed information after receiving the event.
-

Working Process

Parent → Child Communication

1. The Parent component retrieves the Patient Record Id.
 2. The Record Id is passed to the Child component through the `@api` property.
 3. The Child component receives the Record Id.
 4. The Child component displays the corresponding Patient details.
-

Child → Parent Communication

1. The user clicks a button in the Child component.
 2. The Child component fires a custom event.
 3. The Parent component listens for the event.
 4. The Parent component updates its displayed information based on the received event.
-

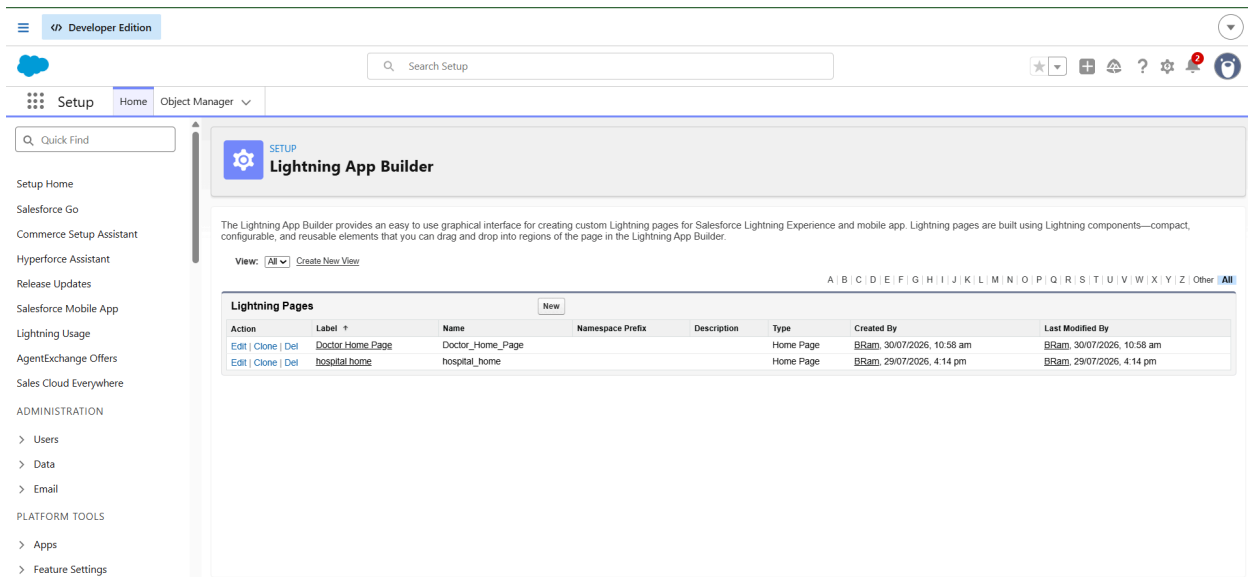
Technologies Used

- Lightning Web Components (LWC)
- JavaScript
- HTML
- XML Configuration
- `@api` Decorator
- Custom Events
- Salesforce CLI
- VS Code
- Lightning App Builder

Learning Outcome

After completing this task, I learned:

- How Parent and Child Lightning Web Components communicate.
- How to expose properties using the `@api` decorator.
- How to send data from Parent to Child.
- How to dispatch and handle custom events.
- How to build reusable and interactive Lightning Web Components.



The screenshot displays the Salesforce Lightning App Builder interface. The top navigation bar includes the 'Developer Edition' label, a search bar, and various utility icons. The left sidebar contains a 'Quick Find' search bar and a list of setup categories: Setup Home, Salesforce Go, Commerce Setup Assistant, Hyperforce Assistant, Release Updates, Salesforce Mobile App, Lightning Usage, AgentExchange Offers, Sales Cloud Everywhere, ADMINISTRATION (Users, Data, Email), and PLATFORM TOOLS (Apps, Feature Settings). The main content area is titled 'Lightning App Builder' and provides an overview of the tool. Below this, a table lists existing Lightning Pages.

Action	Label	Name	Namespace Prefix	Description	Type	Created By	Last Modified By
Edit Clone Del	Doctor Home Page	Doctor_Home_Page			Home Page	BBam, 30/07/2026, 10:58 am	BBam, 30/07/2026, 10:58 am
Edit Clone Del	hospital home	hospital_home			Home Page	BBam, 29/07/2026, 4:14 pm	BBam, 29/07/2026, 4:14 pm

Developer Edition

Lightning App BuilderPagesDoctor Home PageHelp

DesktopShrink To View

Activation...Save

Components

Search...

Standard (57)

Accordion

AI Relationship Research

App Launcher

Assistant

Brand Center Landing Page

Cdp Educational Guidance

Chatter Feed

Chatter Publisher

constraints_studio:constraintsStudi...

constraints_studio:constraintsStudi...

constraints_studio:constraintsStudi...

constraints_studio:constraintsStudi...

Content Search Container

CRM Analytics Collection

Get more on AppExchange

Parent Component

Doctor Name

Dr. Bhanu

Message from Child:No message received yet

Child Component

Doctor Name from Parent:Dr. Bhanu

Send Message to Parent

Page

* Label

Doctor Home Page

* API Name

Doctor_Home_Page

* Page Type

Home Page

Template

Home Template One RegionChange

Description

Developer Edition

Search...

Star

+

Cloud

?

Settings

Notifications

Profile

Sales

Home

Opportunities

Leads

Tasks

Files

Accounts

Contacts

Campaigns

Dashboards

Reports

Chatter

Groups

Calendar

More

Parent Component

Doctor Name

Dr. Bhanu

Message from Child:Hello Parent! Message received from Child.

Child Component

Doctor Name from Parent:Dr. Bhanu

Send Message to Parent